

Project Tracker

Screen Design Specification

Project List, Edit Mode & View Mode — handoff brief for UI/UX design

Prepared for: Product designer (visual / interaction design)

Prepared by: Harsha Shettigar (product owner)

Version: 0.1 — Draft for design

Status: Concept locked in PowerPoint; this document translates those three screens into a buildable spec.

Contents

Contents	2
1. Purpose & how to use this document	3
2. Product context & design principles	3
3. Global UI framework	3
4. Screen 1 — Project List	4
4.1 Purpose	4
4.2 Layout	4
4.3 The project table.....	4
4.4 Create new project (+)	5
4.5 Filter & search row.....	5
5. Screen 2 — Project Detail (View Mode)	5
5.1 Purpose	5
5.2 Layout, top to bottom	5
5.3 The update / history interaction (key behaviour)	6
5.4 What is hidden in View mode.....	6
6. Screen 3 — Project Detail (Edit Mode).....	6
6.1 Purpose	6
6.2 What becomes editable	6
6.3 Adding milestones and tasks	6
6.4 Posting an update.....	7
7. Cross-cutting feature specifications.....	7
7.1 Derived target date	7
7.2 Task & milestone hierarchy and date rules	7
7.3 Status vocabulary	8
7.4 Sub-projects	8
7.5 File attachments & in-app viewing	8
7.6 Create file — built-in spreadsheet.....	8
7.7 Update history pattern (shared)	9
8. Component inventory for the designer.....	9
9. States, empty states & edge cases	10
10. Build & data-model notes (for the dev team)	10
11. Open questions / decisions needed	11

1. Purpose & how to use this document

This document turns the three screens mocked up in the PowerPoint into a clear specification a designer can work from. It describes what each screen is for, the elements it contains, how those elements behave, and the rules that govern data entry. It is intentionally implementation-light: it tells the designer what must be on screen and how it should respond, not how to build it.

The product is a project tracker with two ways to open any project: **View mode** (read-only, for reviews and stakeholders) and **Edit mode** (for the owner to add and update milestones, tasks and files). A single toggle switches between them.

How to read it: Sections 3–6 describe the global frame and the three screens in order. Section 7 collects the cross-cutting rules (date logic, hierarchy, files, spreadsheet, update history) in one place so they aren't lost inside individual screens. Sections 8–9 give the designer a component checklist and the empty / edge states to design for. Sections 10–11 are notes for the build team and a short list of decisions still open.

2. Product context & design principles

Projects are made of milestones and tasks. A project can also contain sub-projects, and can carry attached files and custom in-app spreadsheets. The interface should stay calm and table-driven — this is a working tool people open many times a day, not a marketing surface.

Design principles

- **Clarity over density.** Dates, status and ownership are the load-bearing information; they must be scannable at a glance in both list and detail views.
- **One project, two lenses.** View and Edit share the same layout so switching between them never disorients the user. Only affordances change, not structure.
- **Stay in the app.** Files (especially PDFs) open inside the application wherever possible, so a review never throws the user out to another tool.
- **Latest first.** The most recent update on any task is what people came to see; it is surfaced and highlighted, with history one click away.
- **Status as colour.** A small, consistent colour vocabulary for status (see Section 8) carries through every screen.

3. Global UI framework

These elements are shared by all screens and should be designed once.

3.1 Top bar

A persistent header band runs across the top of every screen. It contains, left to right:

- **App / context title.** “Project Tracker” on the list screen; the project name path on detail screens (and, inside a sub-project, the breadcrumb back to the parent — see 7.4).

- **View / Edit toggle (detail screens only).** A two-state segmented control: an eye (View) and a pencil (Edit). The active state is filled; the inactive is outlined. This is the single control that flips the whole screen between read-only and editable.
- **Create (+) button (list screen).** Opens the new-project flow (see 4.4).
- **Account menu.** Avatar / profile icon at far right; opens user menu (profile, sign out).

Note for designer: in the mockup the toggle, + and account icons share the right edge. Give them clear separation and a fixed order so muscle memory holds across screens.

3.2 Navigation model

- Project list is the home screen and the entry point to everything.
- From a project row, the user chooses Edit or View to open the detail screen in that mode; the toggle then lets them switch without going back.
- Sub-projects are reached only from inside their parent project (Section 7.4). There is no global sub-project list.

4. Screen 1 — Project List

4.1 Purpose

The launchpad. Shows every project the user can see in one scannable table, with the controls to find a project, open it, or start a new one.

4.2 Layout

Top bar, then a filter / search row, then a full-width project table. The table is the focus of the screen.

4.3 The project table

One row per project. Columns:

SI	Project Name	Start Date	Target Date	Status	Responsible
1	Vanillin Development	01/07/2025	01/07/2025	In progress	Harsha
2	MEHQ Development	01/07/2025	01/07/2025	In progress	Amit Sharma
3	Emulsion Product	01/07/2025	01/07/2025	In progress	Nagabhushan

- **Target Date** is read-only here — it is derived, not typed (see 7.1).
- **Status** renders as a coloured chip, not plain text, using the status colours in Section 8.
- **Responsible** is the project owner; show name (and optionally a small avatar).

4.3.1 Row actions — Edit and View

Each row exposes two actions: a pencil (open in Edit mode) and an eye (open in View mode). In the mockup these sit under the Project Name header; for production, place them as a consistent action pair at the end of each row (or revealed on row hover / tap), so every project opens the same way.

4.4 Create new project (+)

The + button in the top bar starts a deliberately minimal new project. The creation form asks for only two things:

1. **Project name**
2. **Start date**

No target date is entered at creation. The target date is **derived later** from the project's contents (see 7.1). On save, the new project appears in the list as a Draft and opens in Edit mode so the owner can begin adding milestones and tasks.

4.5 Filter & search row

Directly above the table, a single row gives the user three ways to narrow the list. This row is new versus the mockup and should be designed in.

- **Owner filter.** Dropdown / multi-select of project owners ("Responsible"). Default: all.
- **Status filter.** Multi-select of statuses (Draft, In progress, On hold, Completed, Cancelled). Default: all.
- **Search.** A short free-text box that matches project name as the user types.

Behaviour: filters and search combine (AND). Show the active filters as removable chips, and a "clear all" affordance. When filters return nothing, show the empty-results state from Section 9.

5. Screen 2 — Project Detail (View Mode)

5.1 Purpose

The read-only face of a project, used for reviews and for sharing with stakeholders. Same layout as Edit mode, but nothing is editable and the reading experience is optimised: latest updates are surfaced and history is one click away.

5.2 Layout, top to bottom

a) Project summary band. Project name, start date, target date, status and responsible owner — the same five fields as the list row, shown as a header band.

b) Objective. A short objective / description paragraph with a "More >" expander when the text is long. Collapsed by default to keep the milestones above the fold.

c) Milestone blocks. Each milestone is a block with its name, target date and status in a coloured header, followed by its task table. Milestones stack in order.

d) Task table (within each milestone). Columns: Task Name, Start Date, Target Date, Status. Under each task sits its latest update (see 5.3).

e) Tasks directly under the project. Tasks that belong to no milestone appear in their own table (e.g. an "Unscheduled / Project tasks" group) using the same columns.

f) Additional Files. A labelled strip of attached files (e.g. Vendor list, Approval list, Spec sheet), each opening in the in-app viewer (Section 7.5).

g) Sub-projects. A labelled strip of links to child projects (Section 7.4).

5.3 The update / history interaction (key behaviour)

This is the defining behaviour of View mode. For every task:

- **The latest update is shown inline and highlighted** immediately beneath the task row (“Update : ...”), so a reviewer sees current status without any clicks.
- **Clicking the task (or its update) expands the full history** — every previous update for that task, newest at top, each stamped with date and author. The latest entry stays visually emphasised.
- Collapsing returns the task to showing only its latest update.

Designer to define: the highlight treatment for the latest update (e.g. tinted background + left emphasis), the expanded history layout (timeline vs. stacked entries), and the open/closed affordance (chevron). Keep the expansion inline so the reviewer never loses their place in the milestone.

5.4 What is hidden in View mode

- No add / edit / delete controls anywhere (no “+ Add task”, no inline editing, no field affordances).
- Files and sub-projects remain openable (read), but not detachable or re-orderable.
- The pencil in the toggle remains available so an owner can switch to Edit; for stakeholder accounts without edit rights, the toggle shows View only.

6. Screen 3 — Project Detail (Edit Mode)

6.1 Purpose

The owner’s workspace. Identical skeleton to View mode, with every region made editable and the controls to add structure (milestones, tasks, files, sub-projects).

6.2 What becomes editable

- **Project summary band:** name, start date, status and owner are editable. Target date stays derived and read-only (7.1).
- **Objective:** editable rich-ish text.
- **Milestones:** add / rename / set target date / set status / reorder / remove.
- **Tasks:** add under a milestone or directly under the project; edit name, start date, target date, status; post an update; reorder; remove.
- **Files:** attach new files; create a new in-app spreadsheet (7.6); remove files.
- **Sub-projects:** add / link a child project; remove.

6.3 Adding milestones and tasks

Provide clear add affordances: “+ Add milestone” at the project level, and “+ Add task” both inside each milestone block and in the project-level task group. Adding should be lightweight (inline row or small modal), not a full page.

Critical entry rules (design the form to enforce these): the target-date requirement differs by where the task lives. See the rules table in 7.2 — these are mandatory and the form should make the required vs. optional fields obvious in context.

6.4 Posting an update

In Edit mode, each task offers an “Add update” action. A posted update becomes the new “latest” (highlighted) entry in View mode and is appended to that task’s history. Updates are additive — posting a new one never overwrites the previous; the prior update moves into history.

7. Cross-cutting feature specifications

These rules apply across screens and are the heart of how the tool behaves. They are gathered here so they are designed and built consistently.

7.1 Derived target date

A project’s target date is never typed; it is computed from the project’s contents and recomputed whenever they change. The rule:

- **Target date = the latest target date among the project’s milestones and its directly-attached tasks.**
- In practice this is usually the target date of the last milestone to complete; if a project has standalone tasks that run later, the latest of those wins.
- It is shown read-only in the list (Screen 1) and in the summary band (Screens 2 & 3), and updates automatically as milestones / tasks are added, removed or re-dated.
- A brand-new project (name + start date only, no contents yet) has no target date — design an empty treatment for this (e.g. “—” or “Not set”).

7.2 Task & milestone hierarchy and date rules

Two ways to organise work, with different date requirements:

Item	Lives under	Target date	Start date
Milestone	The project	Required	Optional
Task under a milestone	A milestone	Optional	Optional
Task directly under project	The project (no milestone)	Required	Optional

- A task can exist without any milestone — the tool supports a flat project of loose tasks as well as a milestone-structured one.

- When a task sits under a milestone, the milestone's target date is the governing deadline, so the task's own target date is optional.
- A task with no milestone has no governing deadline of its own, so its target date is required.

Design implication: the same “add task” form must change its required fields depending on context. Inside a milestone, target date is optional; at project level, it is required. Make this obvious (required marker, inline validation) rather than letting the user discover it on save.

7.3 Status vocabulary

Use one status set consistently for projects, milestones and tasks (wording can differ slightly per level, but the colour meaning must not). Suggested set and colours are in Section 8.

7.4 Sub-projects

- **A project can contain sub-projects.** A sub-project is itself a full project and opens into the same detail screen, with the same View / Edit modes.
- **Access is only through the parent.** Sub-projects are listed in the parent's detail screen (the “Sub Projects” strip) and are not shown on the top-level project list.
- **Breadcrumb.** When inside a sub-project, the top bar shows the path back to the parent (e.g. “Vanillin Development › Vanillin Market Research”) so the user can always navigate up.

Designer to define: how deep nesting can go (recommend a visible limit, e.g. one level, unless the team needs more), and how a sub-project's status / dates roll up (if at all) into the parent's view.

7.5 File attachments & in-app viewing

Files are attached to a project (the “Additional Files” strip) and, where useful, to milestones or tasks.

- **Open in-app first.** Clicking a file opens it inside the application. PDFs in particular must render in an in-app viewer so the user never leaves the tool during a review.
- **Graceful fallback.** If the application cannot render a given file type, it downloads the file and then opens it for reading — the user still reaches the content, just via download.
- **Upload constraints (from the security checklist).** Allowed types are limited to PDF, PNG, JPG, DOCX and XLSX; uploads are size-capped (around 25 MB) and scanned before they are stored. Design the upload control and its error states (wrong type, too large) accordingly.

Designer to define: the viewer chrome (a modal / side panel / full-screen reader), how a file list entry looks (icon by type, name, size, who attached it), and the download-fallback messaging.

7.6 Create file — built-in spreadsheet

Alongside attaching external files, the owner can create a custom, viewable file inside the app. Version 1 of this is a built-in spreadsheet with the basics a working team needs:

- Create a basic table / grid; enter and edit cell values.

- Lists within the sheet.
- Merge cells.
- Sum (and comparable simple aggregate formulas).
- Pivot-style summarisation of the table.

The created sheet behaves like any other file in the project: it appears in the file strip and opens in-app for viewing (and editing, in Edit mode).

Designer to define: the spreadsheet shell (toolbar with the above actions), how a sheet is named and saved, and how view-only rendering differs from editable. This is the largest single new surface in the product — worth its own detailed screen spec after this document is approved.

7.7 Update history pattern (shared)

The latest-update-highlighted, click-to-expand-history pattern described in 5.3 is the canonical way updates are shown anywhere in the product. Design it once as a reusable component (a “task update thread”) and reuse it for tasks under milestones and tasks under the project alike.

8. Component inventory for the designer

The minimum set of reusable components these three screens require:

Component	Notes
Top bar	Title / breadcrumb, View-Edit toggle, + Create, account menu.
View / Edit toggle	Two-state segmented control (eye / pencil); active state filled.
Filter & search row	Owner select, status multi-select, search box, active-filter chips, clear-all.
Project table row	SI, name, dates, status chip, owner; row-level Edit / View actions.
Status chip	Coloured pill; one vocabulary across project / milestone / task.
Summary band	The five project header fields in detail screens.
Milestone block	Coloured header (name, target date, status) + task table.
Task row	Name, start, target, status; expandable to the update thread.
Update thread	Latest update highlighted; expand to dated, authored history.
File strip + file entry	Type icon, name, size; opens in-app viewer.
In-app file viewer	Modal / panel; PDF renderer + download-fallback path.
Spreadsheet shell	Grid, toolbar (table, list, merge, sum, pivot), view vs. edit.
Sub-project strip / link	Links to child projects with breadcrumb on entry.
Add affordances	+ Add milestone, + Add task (context-aware required fields).
Empty / error states	See Section 9.

Suggested status colours

A small, consistent palette (designer may refine the exact hues for accessibility / contrast):

- **Draft** — neutral grey
- **In progress** — amber
- **Completed / Done** — green
- **At risk / Blocked** — red
- **On hold** — muted blue

9. States, empty states & edge cases

- **No projects yet.** First-run list screen: friendly empty state with a prominent Create-project call to action.
- **Filters / search return nothing.** “No projects match” with a clear-filters action.
- **New project, no contents.** Target date shows “— / Not set”; detail screen prompts the owner to add a first milestone or task.
- **Milestone with no tasks.** Show the milestone header with an empty task table and “+ Add task”.
- **Task with no updates yet.** Show “No updates yet” in place of the highlighted latest-update line.
- **Unsupported file type.** Download-and-open fallback with a brief explanatory message.
- **Upload rejected.** Clear inline error for wrong type or oversize file.
- **View-only user.** Toggle hides the Edit option; all add / edit controls absent.
- **Long objective text.** Collapse behind “More >” so milestones stay above the fold.

10. Build & data-model notes (for the dev team)

Not for the designer, but worth recording so design and build stay aligned. Three things in this spec are not yet represented in the current database schema and will need to be added:

- **Sub-projects.** Projects need a self-reference (parent project) to support the sub-project relationship in 7.4.
- **File attachments.** There is no attachment entity yet; one is needed to store uploaded files (and link them to project / milestone / task), with the type and size constraints from the security checklist.
- **In-app spreadsheets.** The “create file” spreadsheet (7.6) needs a place to persist sheet data; this is genuinely new scope and the biggest build item here.

Two things map cleanly onto what already exists: tasks without a milestone are already supported (a task’s milestone link is optional), and the task update history can build on the existing comment / activity records. The derived target date (7.1) is application logic, not a schema change.

11. Open questions / decisions needed

3. Sub-project depth: one level only, or arbitrary nesting? Does a sub-project's status / dates roll up into the parent's view?
4. Update authorship & permissions: who can post a task update — owner only, or any assigned member?
5. Spreadsheet scope for v1: is the 7.6 list (table, list, merge, sum, pivot) the firm v1 boundary, or are formulas / charts expected?
6. File viewer surface: modal, side panel, or full-screen reader?
7. Status wording per level: keep identical labels for project / milestone / task, or tailor them?
8. Target-date derivation when a project has both later standalone tasks and earlier milestones — confirm “latest of all” is the intended rule.